

# Ansible Core Concepts

Q1. Explain the following briefly: (a) Playbooks (b) Inventory (c) Modules

## (a) Playbooks

*The Recipe for Automation*

A **Playbook** is a YAML file that defines exactly what Ansible should do and on which machines. Think of it as a recipe — you list the ingredients (tasks) and steps in order. Each task calls a module to perform a specific action such as installing software, copying files, or restarting a service. Playbooks are human-readable, so even someone new to the project can understand what a playbook does just by reading it.

**Example use:** *Install Nginx → deploy app → restart service — all in one playbook.*

## (b) Inventory

*The Address Book of Your Infrastructure*

An **Inventory** is a file (or dynamic source) that lists all the machines Ansible is allowed to manage. Without an inventory, Ansible simply doesn't know where to connect. Hosts can be grouped logically — for example, `[webservers]`, `[databases]`, or `[staging]` — and a playbook can then target one or more of these groups. Inventories can be static (a plain `.ini` or `.yaml` file) or dynamic, pulled live from cloud providers like AWS, Azure, or GCP.

**Example use:** *Target only `[webservers]` group to deploy a new config update.*

## (c) Modules

*The Actual Workers*

**Modules** are small, focused units of code that each perform a single task. Every time Ansible runs a task in a playbook, it calls a module under the hood. Ansible ships with 3,000+ built-in modules covering package management (`apt`, `yum`), file operations (`copy`, `template`), cloud provisioning, and database management. Modules are idempotent — running them multiple times produces the same result, so re-running a playbook won't accidentally break a system already in the desired state.

**Common modules:** `apt`, `yum`, `copy`, `service`, `template`, `user`, `git`, `aws_ec2`.

### Playbooks

*What to do*

### Inventory

*Where to run*

### Modules

*How each task runs*